



F107 – Foundations of Computer Science

Lecture 3

Dr. Sanjit Kumar Saha

Topics

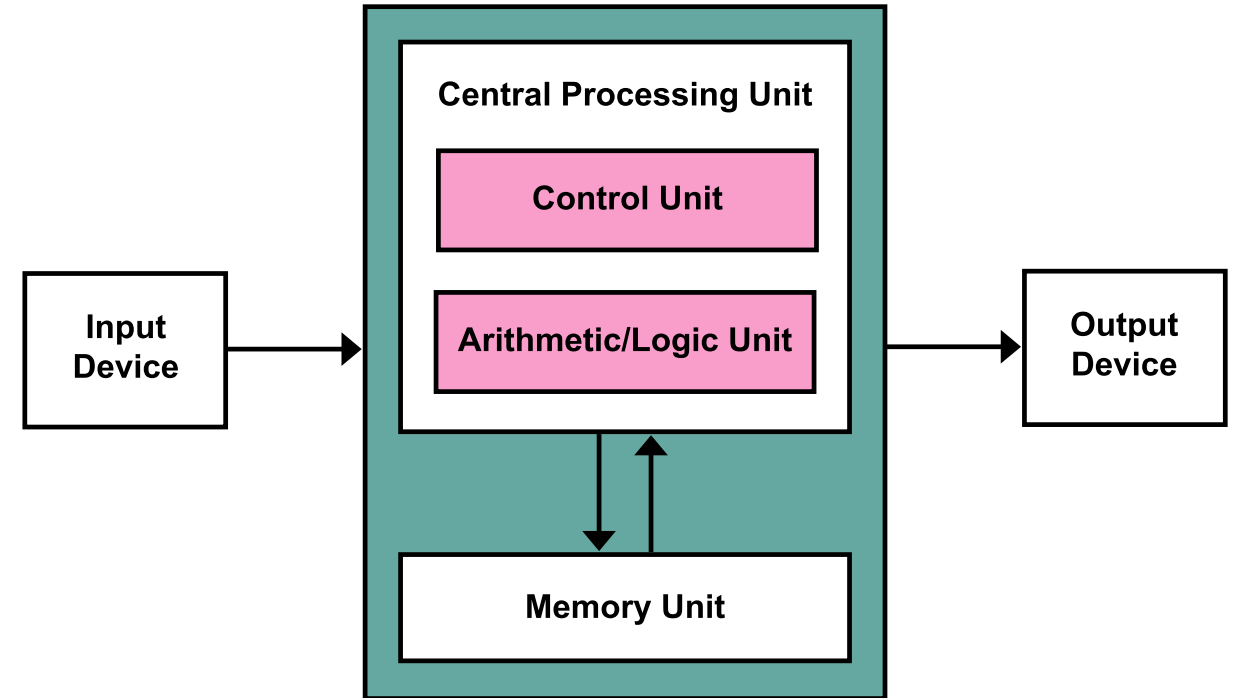


- Computer Architecture
 - CPU
 - Memory
 - Bus systems
 - Inputs and Outputs
 - Fetch-Decode-Execute Approach
 - Special Topics
 - Parallelization and pipelining
 - Direct Memory Access
 - Interrupts and Exceptions
 - Multi-core CPUs
- “Foundations of Computer Science” by B. Forouzan
 - Chapter 5

Introduction to Computer Architecture



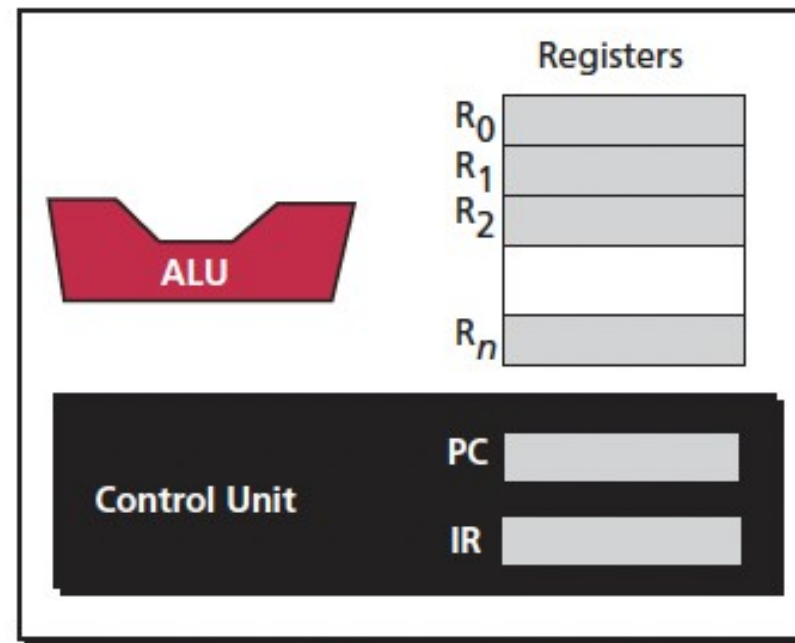
- Von-Neumann-Architecture model
 - CPU
 - Memory
 - Inputs and Outputs
 - Other components
- Application scenario defines
 - Form factor (size, weight)
 - Computational capabilities
 - Ability to exchange components



Central Processing Unit (CPU)



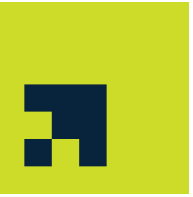
- The CPU performs operations on data.
- The brain of the computer
 - Executes instructions
 - Controls interactions with memory and peripherals
- Three parts:
 - an arithmetic logic unit (ALU),
 - a control unit, and
 - a set of registers,



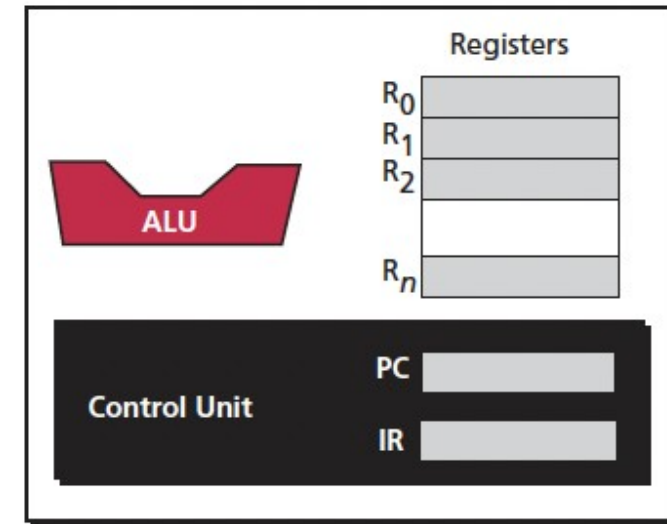
R: Register
PC: Program Counter
IR: Instruction Register

Central Processing Unit (CPU)

Arithmetic Logic Unit (CPU)

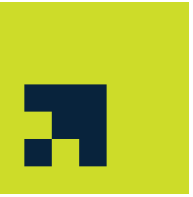


- The arithmetic logic unit (ALU) performs logic, shift, and arithmetic operations on data.
 - **Logic operations**
 - such as NOT, AND, OR, and XOR.
 - treat the input data as bit patterns and the result of the operation is also a bit pattern.
 - **Shift operations**
 - logical shift operations: shift bit patterns to the left or right,
 - arithmetic shift operations: are applied to integers to divide or multiply integers by two.
 - **Arithmetic operation:** Addition, Subtraction, Multiplication, and Division

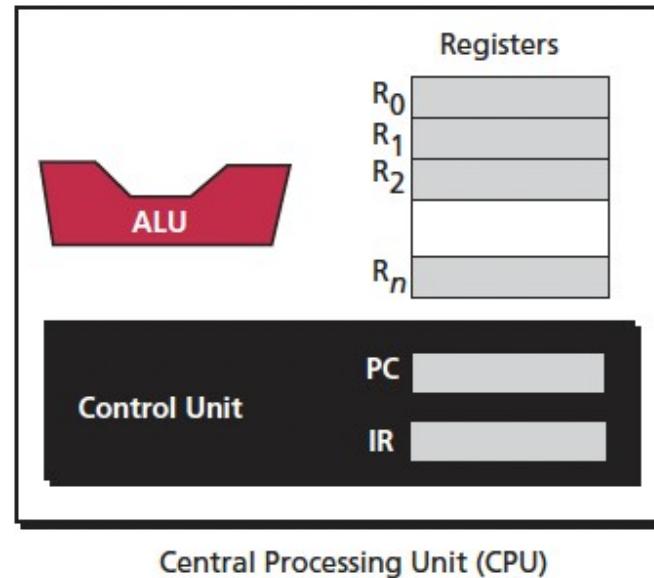


Central Processing Unit (CPU)

Control Unit



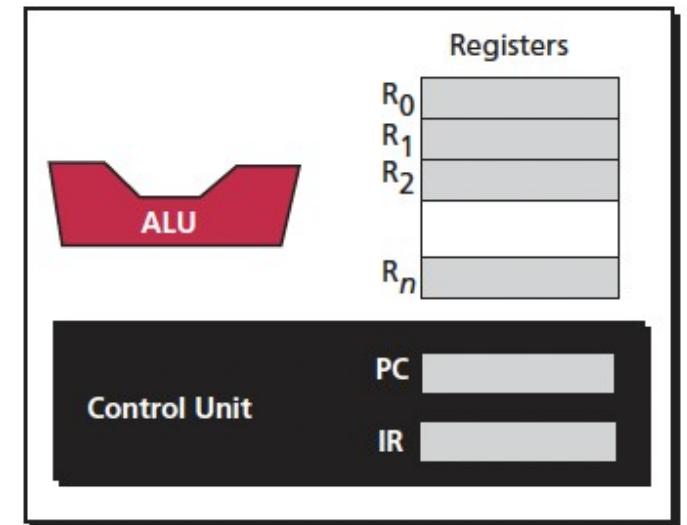
- The control unit controls the operation of each subsystem.
- Controlling is achieved through signals sent from the control unit to other subsystems.



Registers

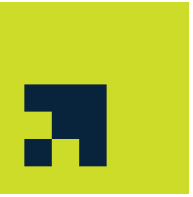


- Registers are fast stand-alone storage locations that hold data temporarily.
- **Data registers**
 - hold the input data and the result of the operations.
 - speed up their operations, because complex operations are done using hardware instead of software.
 - named R_1 to R_n .
- **Instruction registers**
 - store instructions one by one of a program
 - fetched by CPU from memory
 - decoding them, and executing them.
- **Program counter**
 - keeps track of the instruction currently being executed.
 - After execution, the counter is incremented to point to the address of the next instruction in memory.

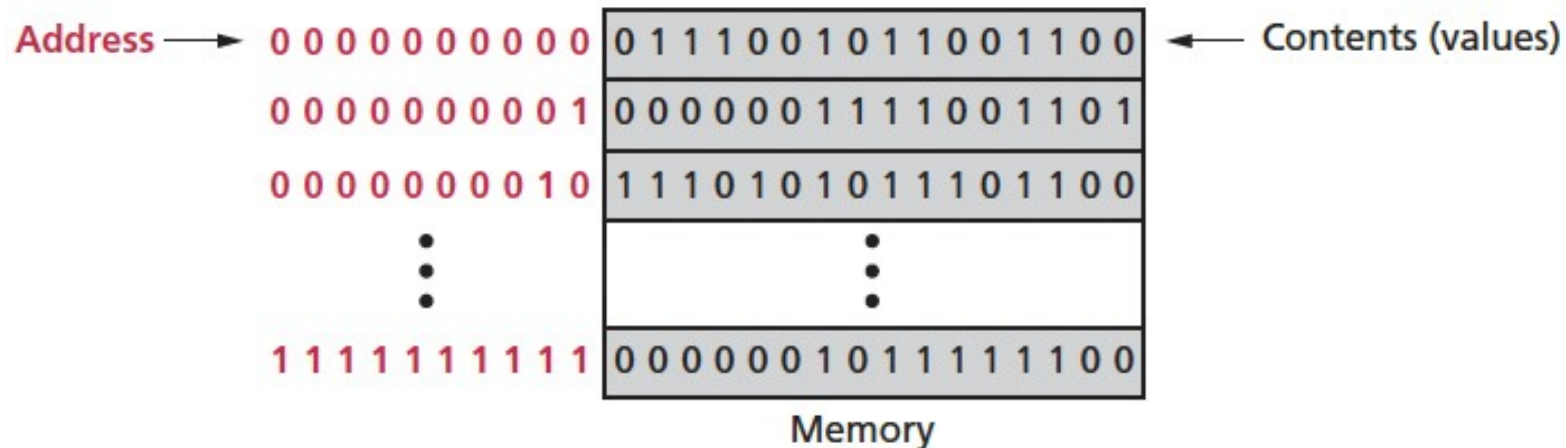


Central Processing Unit (CPU)

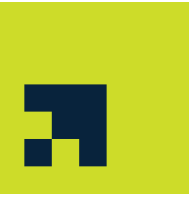
Main Memory



- It consists of a collection of storage locations, each with a unique identifier, called an address.
- Data is transferred to and from memory in groups of bits called words.
- A word can be a group of 8 bits, 16 bits, 32 bits, or 64 bits (and growing).
- If the word is 8 bits, it is referred to as a byte.
- Sometimes a 16-bit word is referred to as a 2-byte word, or a 32-bit word is referred to as a 4-byte word.



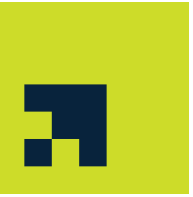
Address space



- To access a word in memory requires an identifier.
- Although programmers use a name to identify a word (or a collection of words), at the hardware level each word is identified by an address.
- The total number of uniquely identifiable locations in memory is called the address space.
- For example, a memory with 64 kilobytes and a word size of 1 byte has an address space that ranges from 0 to 65 535.

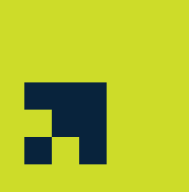
<i>Unit</i>	<i>Exact Number of Bytes</i>	<i>Approximation</i>
kilobyte	2^{10} (1024) bytes	10^3 bytes
megabyte	2^{20} (1 048 576) bytes	10^6 bytes
gigabyte	2^{30} (1 073 741 824) bytes	10^9 bytes
terabyte	2^{40} bytes	10^{12} bytes

Addresses as bit patterns



- Because computers operate by storing numbers as bit patterns, a memory address is also represented as a bit pattern.
- So if a computer has 64 kilobytes (2^{16}) of memory with a word size of 1 byte, we need a bit pattern of 16 bits to define an address.
 - the first location is referred to as address 0000000000000000 (address 0), and the last location is referred to as address 1111111111111111 (address 65535).
- In general, if a computer has N words of memory, we need an unsigned integer of size $\log_2 N$ bits to refer to each memory location.

Example



- A computer has 32 MB (megabytes) of memory. How many bits are needed to address any single byte in memory?

Solution:

The memory address space is 32 MB, or 2^{25} ($2^5 \times 2^{20}$). This means that we need $\log_2 2^{25}$, or 25 bits, to address each byte.

Memory Types



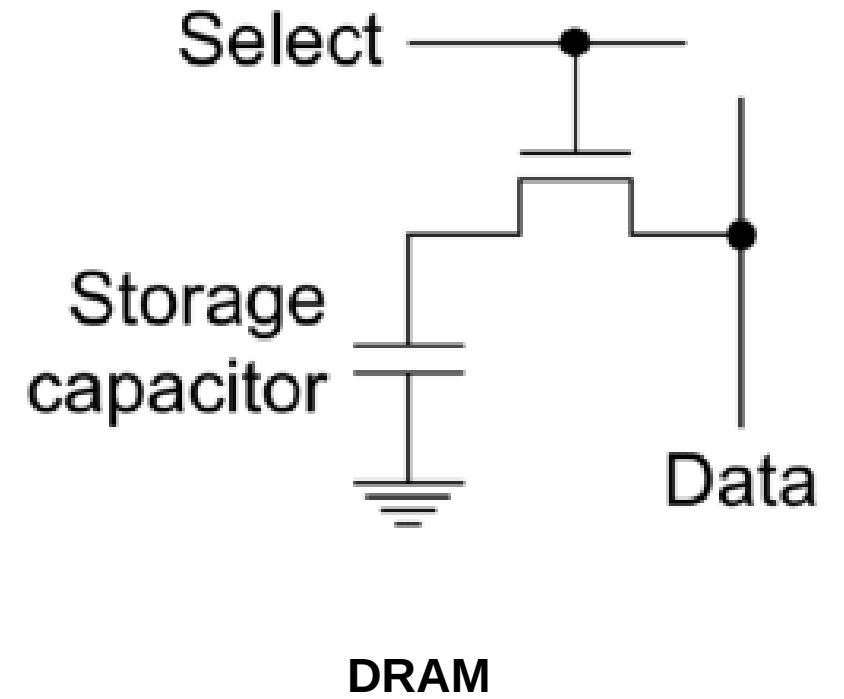
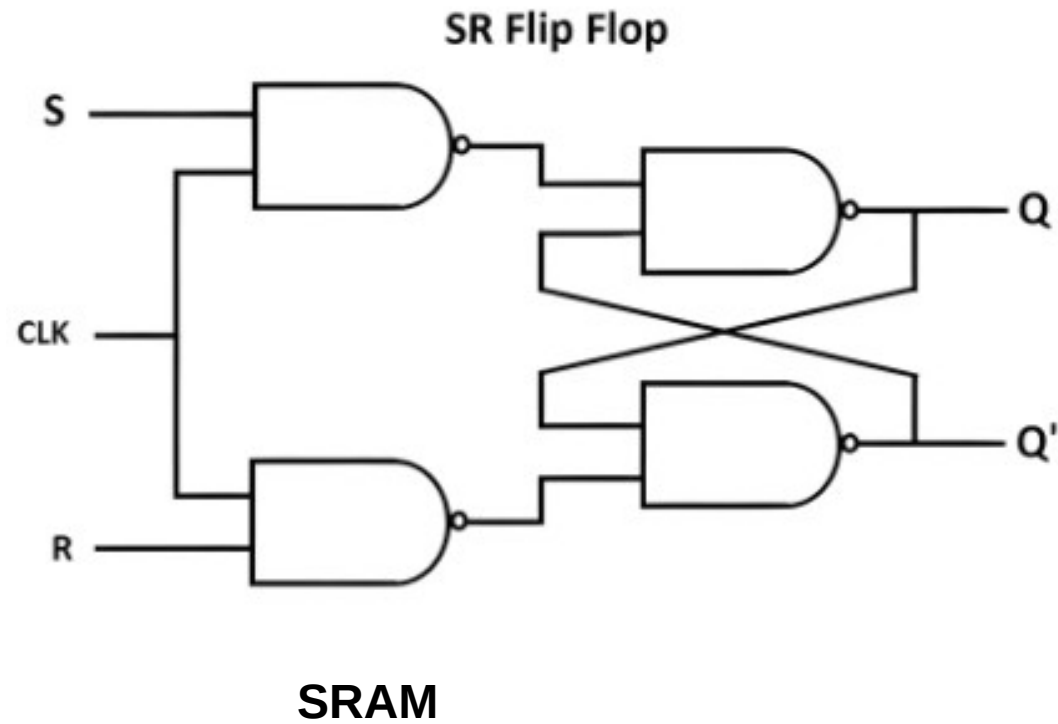
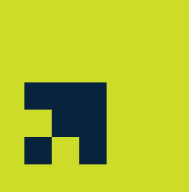
- RAM: Random Access Memory
- ROM: Read Only Memory

RAM



- It makes up most of the main memory in a computer.
- a data item can be accessed randomly—using the address of the memory location—without the need to access all data items located before it.
- The CPU can write something to RAM and later overwrite it.
- RAM is volatile.
- RAM categories:
 - SRAM: uses traditional flip-flop gates to hold data as long as power is on; no need to refresh; fast and expensive.
 - DRAM: uses capacitors, electrical devices that can store energy, for data storage; need to be refreshed periodically; slow but inexpensive.

SRAM vs DRAM



ROM

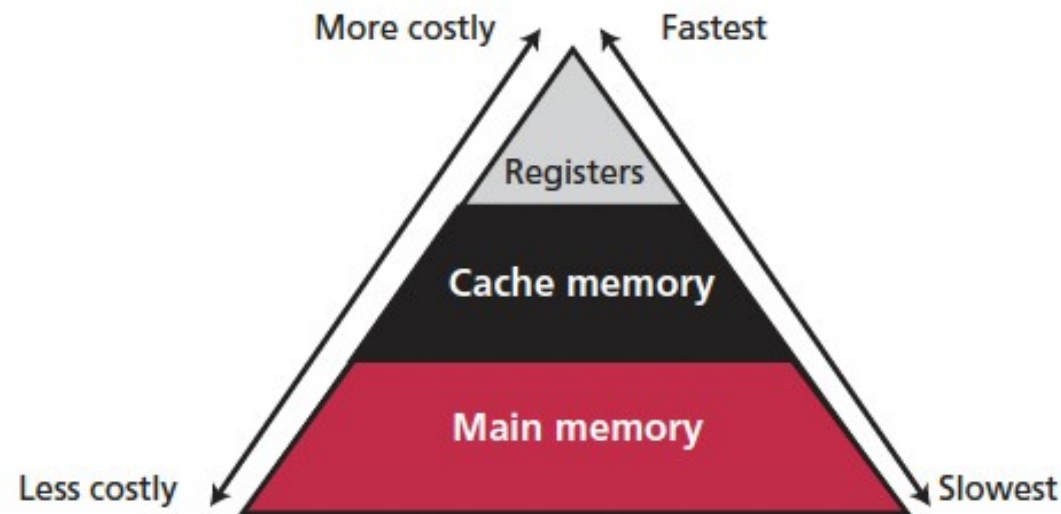


- The CPU can read from, but not write to, ROM.
- It is nonvolatile.
- It is used for programs or data that must not be erased or changed even if you turn off the computer.
- For example, some computers come with ROM that holds the boot program that runs when we switch on the computer.
- **PROM**: initially blank; can store programs with some special equipment; can not overwritten
- **EPROM**: programmed by the user; can also be erased with a special device that applies ultraviolet light; to erase it requires physical removal and reinstallation.
- **EEPROM**: programmed and erased using electronic impulses without being removed.

Memory hierarchy



- Use a very small amount of costly high-speed memory where speed is crucial. The registers inside the CPU are of this type.
- Use a moderate amount of medium-speed memory to store data that is accessed often. Cache memory is of this type.
- Use a large amount of low-speed memory for data that is accessed less often. Main memory is of this type.



Memory hierarchy



- Organized by speed, cost, and size
 - From registers (fastest) to secondary storage (slowest)
 - Improves performance by reducing memory access delays
- Technical realization
 - Tier 0 – 2/3: Electrical
 - Tier 3: Sometimes magnetic
 - Tier 4: Optical or magnetic
- Storage is imperfect
 - Errors may occur over time
 - Detection/correction mechanisms required

Tier 0

- CPU: Register

Tier 1

- CPU: Cache memory (various levels)

Tier 2

- RAM: Main memory

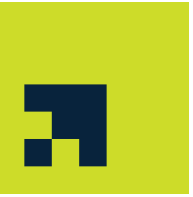
Tier 3

- Storage: HDD or SSD

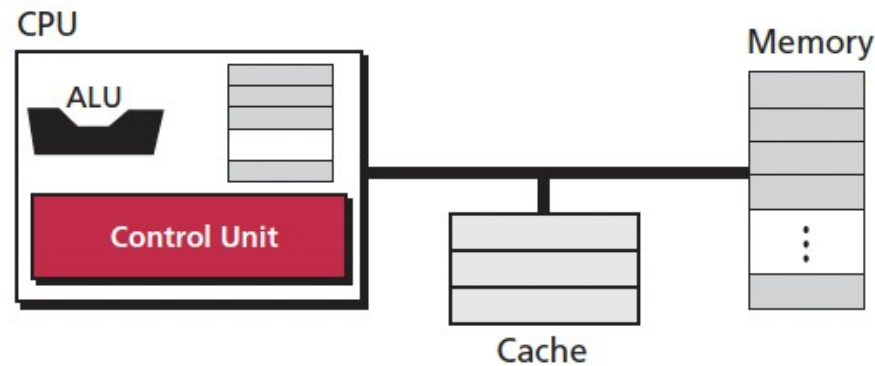
Tier 4

- Archiving: Optical disks or magnetic tapes

Cache Memory



- Cache memory is faster than main memory but slower than the CPU and its registers.
 - normally small in size, is placed between the CPU and main memory



- When the CPU needs to access a word in main memory, it follows this procedure:
 1. The CPU checks the cache.
 2. If the word is there, it copies the word: if not, the CPU accesses main memory and copies a block of memory starting with the desired word. This block replaces the previous contents of cache memory.
 3. The CPU accesses the cache and copies the word.

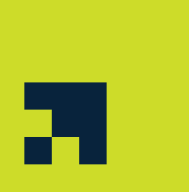
Input and Output (I/O) Devices



- Facilitate communication between computer and external world
 - Can be Input-only, Output-only, Input-Output
 - CPU interfaces with them, using addresses
- Classifications done by
 - I/O direction
 - Permanent vs. temporary
 - Medium
- Interconnection with CPU via busses
 - Isolated I/O approach
 - Special instructions for I/O, address ranges do not matter
 - Memory-mapped approach
 - Identical instructions, careful addressing required

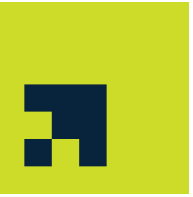


System Bus

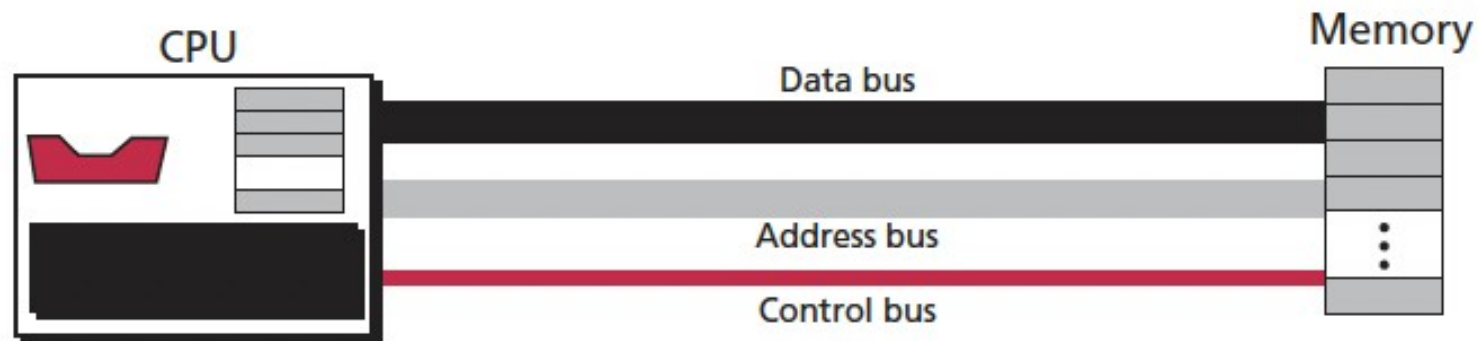


- Buses are communication pathways between components
 - Fixed or extendable
 - On mainboard, with cables, with plugs
 - Unidirectional vs. bidirectional
 - Serial vs. parallel
- Classes of busses
 - Data bus
 - Transfers data words
 - Address bus
 - Carries address words
 - Control bus
 - Simple signals to instruct devices
 - Clock signals

Connecting CPU and memory



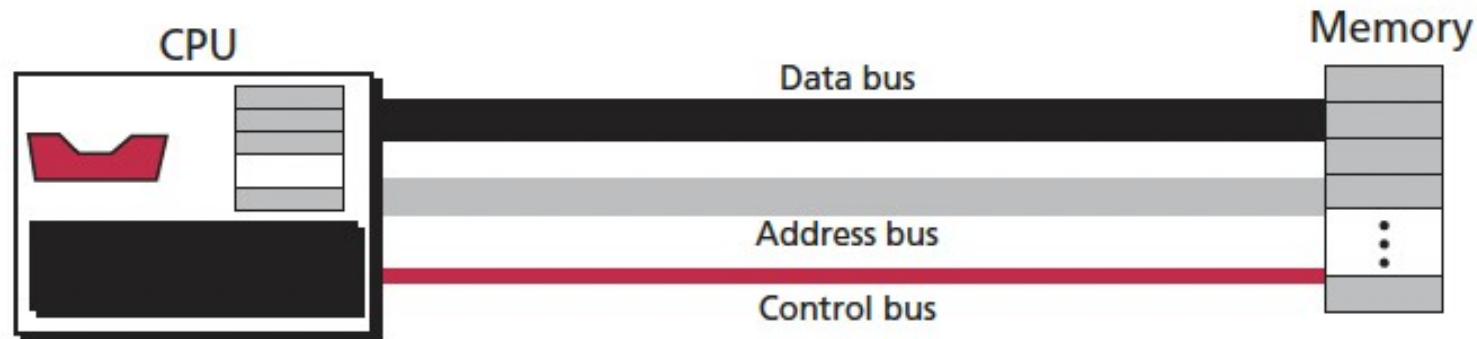
- Connected by three groups of connections, each called a bus:
 - Data bus
 - Transfers data words
 - Address bus
 - Carries address words
 - Control bus
 - Simple signals to instruct devices
 - Clock signals



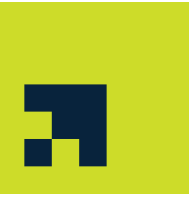
Data bus



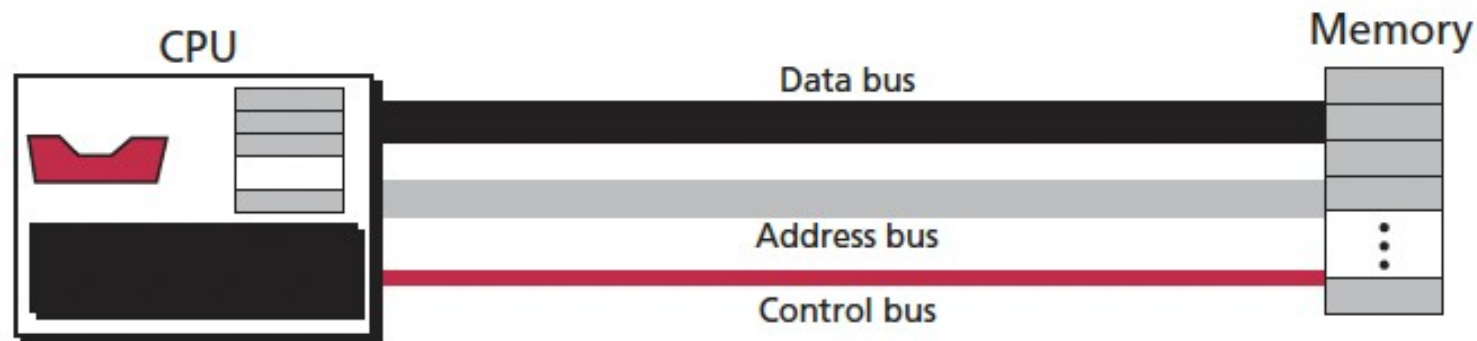
- The data bus is made of several connections, each carrying 1 bit at a time.
- The number of connections depends on the size of the word used by the computer.
- If the word is 32 bits (4 bytes), we need a data bus with 32 connections so that all 32 bits of a word can be transmitted at the same time.



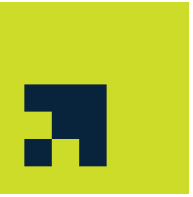
Address bus



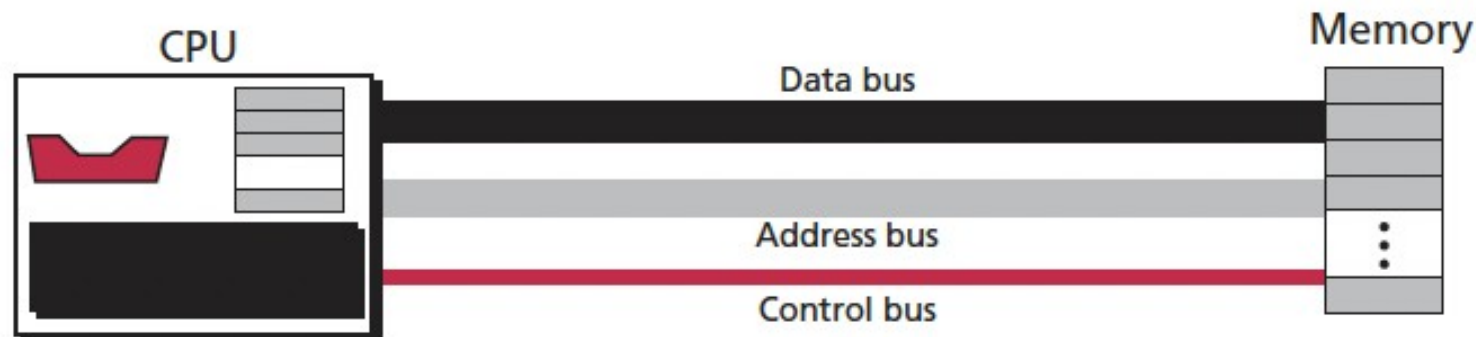
- The address bus allows access to a particular word in memory.
- The number of connections in the address bus depends on the address space of the memory.
- If the memory has 2^n words, the address bus needs to carry n bits at a time. Therefore, it must have n connections.



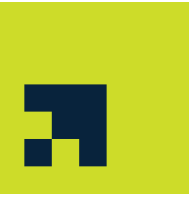
Control bus



- The control bus carries communication between the CPU and memory.
- For example, there must be a code, sent from the CPU to memory, to specify a read or write operation.
- The number of connections used in the control bus depends on the total number of control commands a computer needs.
- If a computer has 2^m control actions, we need m connections for the control bus, because m bits can define 2^m different operations.



Connecting I/O Devices

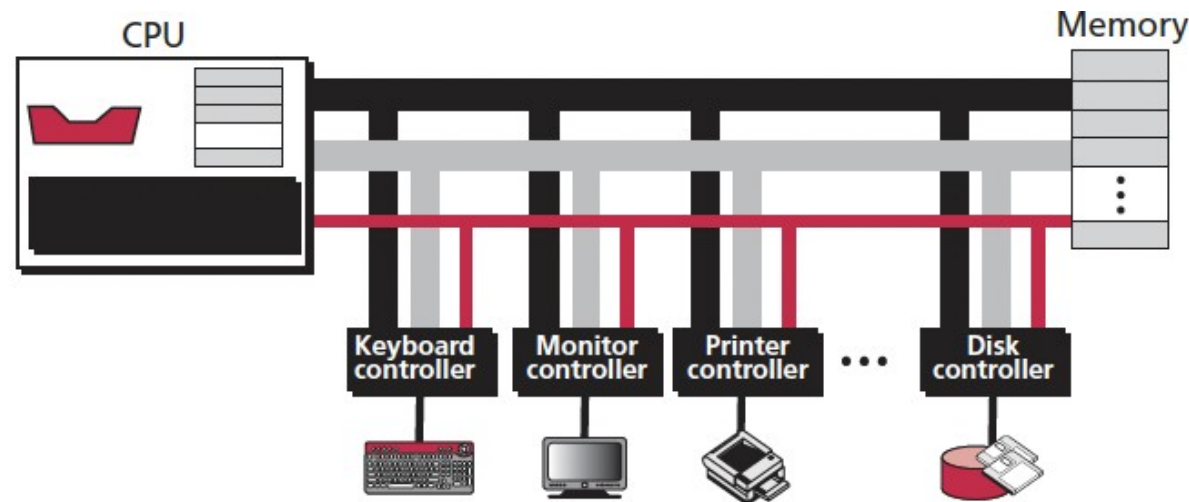


- I/O devices cannot be connected directly to the buses.
 - I/O devices are electromechanical, magnetic, or optical devices, whereas the CPU and memory are electronic devices.
 - I/O devices also operate at a much slower speed than the CPU/memory.
- There is a need for some sort of intermediary to handle this difference.
- Input/output devices are therefore attached to the buses through input/output controllers or interfaces.
- There is one specific controller for each input/output device.

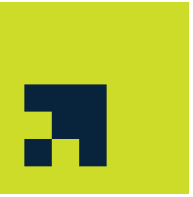
Controllers



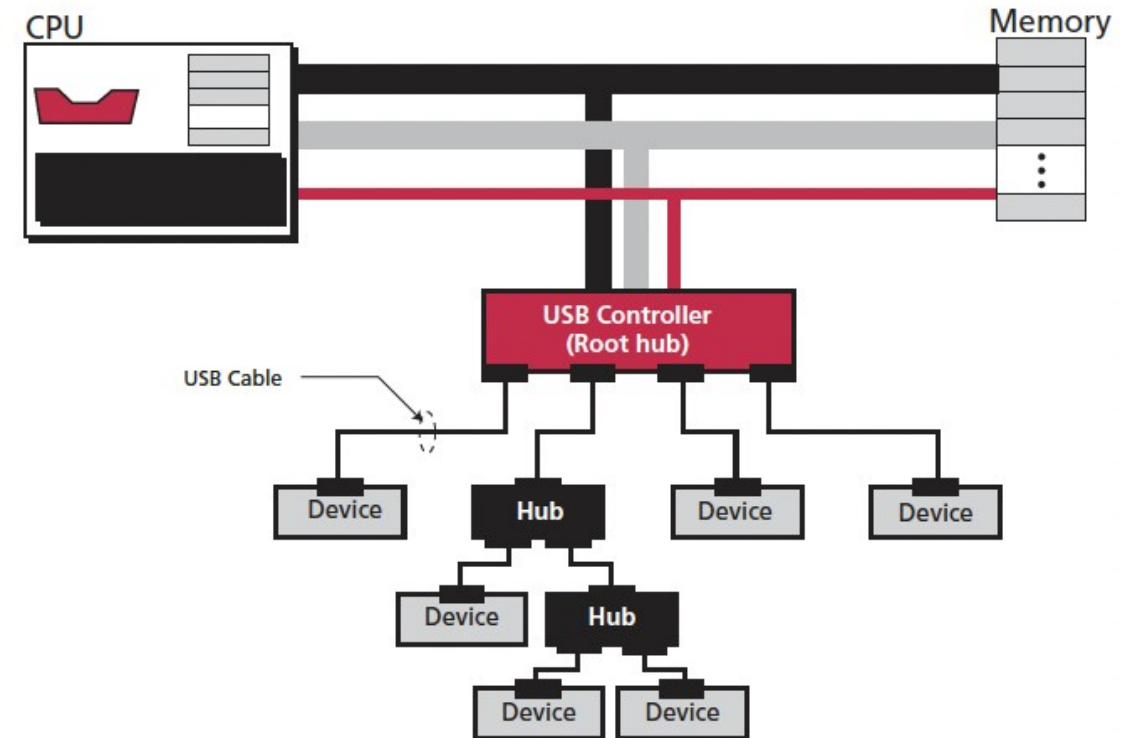
- Controllers or interfaces bridge the gap between the nature of the I/O device and the CPU and memory.
- A controller can be a serial or parallel device.
- A serial controller has only one data wire, while a parallel controller has several data connections so that several bits can be transferred at a time.
- Several kinds of controllers are: SCSI (small computer system interface), FireWire, USB, and HDMI.



Universal Serial Bus (USB)



- USB is a serial controller that connects both low- and high-speed devices to the computer bus.
- Multiple devices can be connected to a USB controller, which is also referred to as a root hub.
- Devices can easily be removed or attached to the tree without powering down the computer.
- USB uses a cable with four wires.
 - Two wires (+5 volts and ground) are used to provide power for low-power devices such as keyboards or mice.
 - The other two wires (twisted together to reduce noise) are used to carry data, addresses, and control signals.



High-Definition Multimedia Interface (HDMI)



- HDMI is a digital replacement for existing analog video standards.
- It can be used for transferring video data digital audio data from a source to a compatible computer monitor, video projector, digital television, or digital audio device.
- There are a number of HDMI-standard cables available including standard, enhanced, high definition, and 3D video signals; up to eight channels of compressed or uncompressed digital audio; a CEC (Consumer Electronics Control) connection; and an Ethernet data connection.

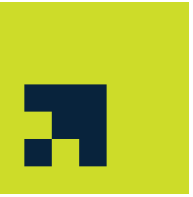
Program Execution



- CPUs execute programs
 - All computer components driven by heartbeat, a “clock” signal
 - CPUs transition from state to state with each pulse of the clock signal
- Instructions are executed via Fetch-Decode-Execute Approach
 - Repeated for every instruction
 - Typically covers one clock cycles, sometimes more
 - Realized mostly in the CU part of the CPU



Fetch-Decode-Execute Approach



- Phase details
 - Fetch instruction:
 - PC is used to fetch next instruction, returned values stored into IR, PC is incremented
 - Decode instruction:
 - Opcode is identified, as well as data and whether operands must be loaded
 - Evaluate address:
 - Addresses of operands are evaluated
 - Fetch operands:
 - Operands are collected from different types of memory and stored in registers
 - Execute operation:
 - ALU executes provided opcode and data
 - Store result:
 - Result is written into a register

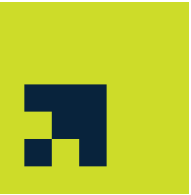


Input/output operation

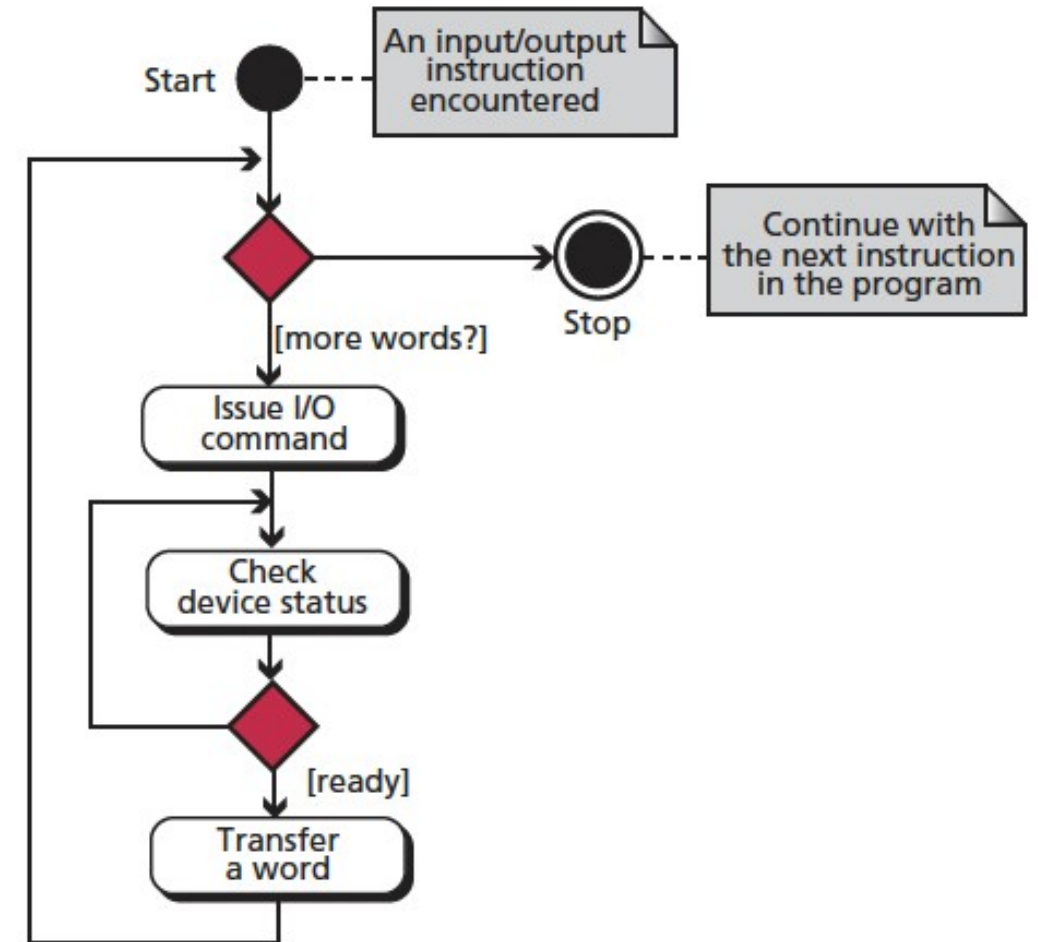


- Three methods:
 - programmed I/O,
 - interrupt-driven I/O, and
 - direct memory access (DMA).

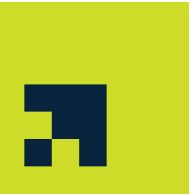
Programmed I/O



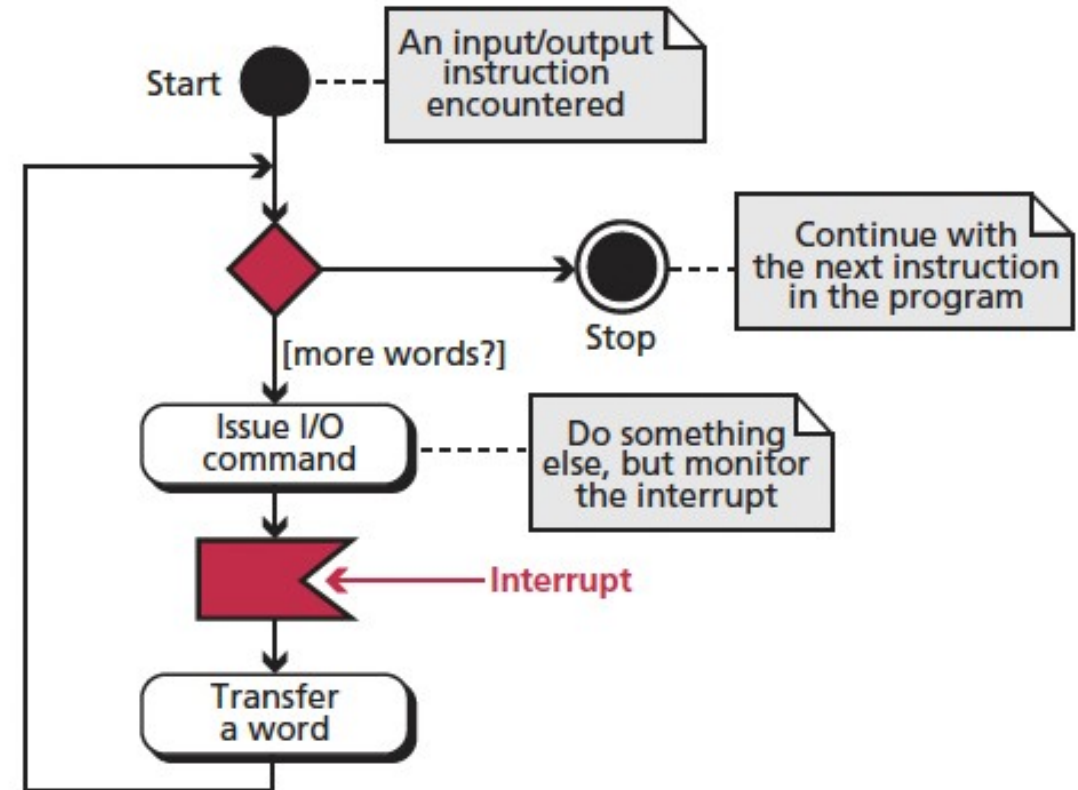
- The CPU waits for the I/O device
- Data transfer is done by an instruction in the program.
- When the CPU encounters an I/O instruction, it does nothing else until the data transfer is complete.
- The CPU constantly checks the status of the I/O device:
 - if the device is ready to transfer, data is transferred to the CPU.
 - if the device is not ready, the CPU continues checking the device status until the I/O device is ready.
- Data is transferred to memory after the input operation, while data is transferred from memory before the output operation.



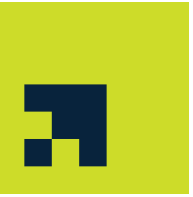
Interrupt-driven I/O



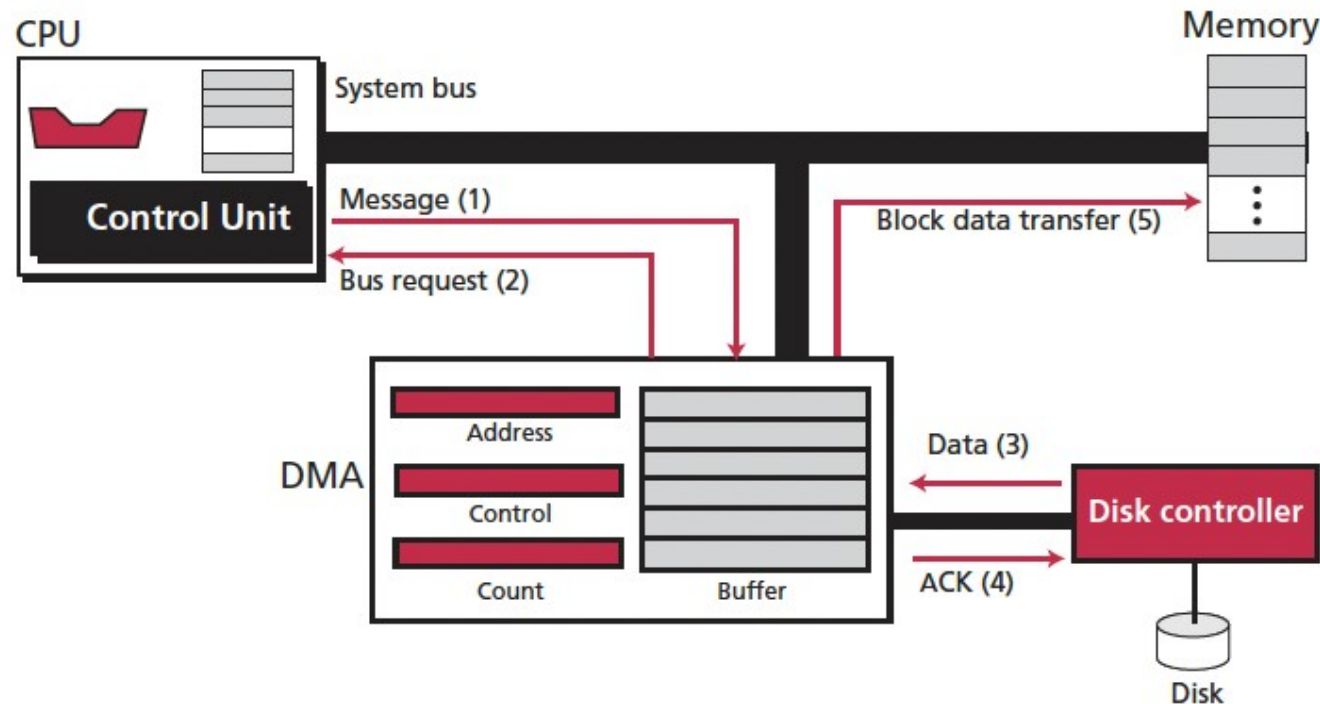
- The CPU informs the I/O device that a transfer is going to happen, but it does not test the status of the I/O device continuously.
- The I/O device informs (interrupts) the CPU when it is ready.
- During this time, the CPU can do other jobs such as running other programs or transferring data from or to other I/O devices.
- Data is transferred to memory after the input operation. while data is transferred from memory before the output operation.



Direct memory access (DMA)



- This method transfers a large block of data between a high-speed I/O device, such as a disk, and memory directly without passing it through the CPU.
- This requires a DMA controller that relieves the CPU of some of its functions.
- The DMA controller has registers to hold a block of data before and after memory transfer.

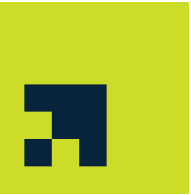


RISC

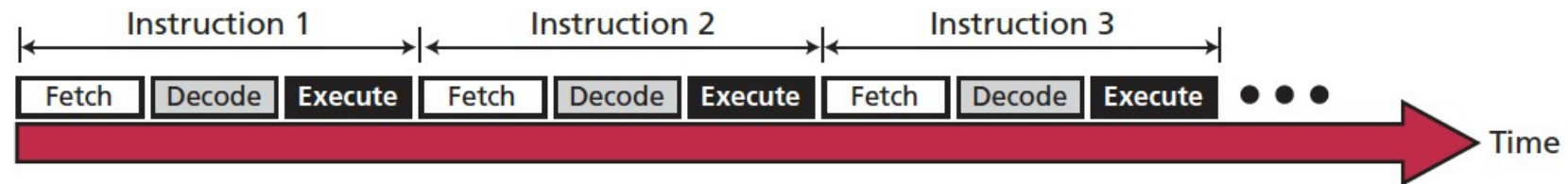


- RISC stands for reduced instruction set computer.
- The strategy behind RISC architecture is to have a small set of instructions that do a minimum number of simple operations.
- Complex instructions are simulated using a subset of simple instructions.
- Programming in RISC is more difficult and time-consuming than in the other design because most of the complex instructions are simulated using simple instructions.

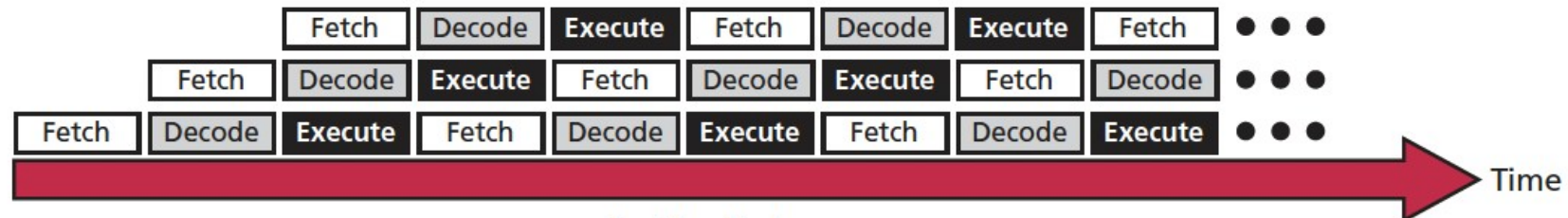
Pipelining



- A computer uses three phases of fetch, decode, and execute for each instruction .
- If the control unit can do two or three of these phases simultaneously, the next instruction can start before the previous one is finished.
- When the CPU is performing the decode phase of the first instruction, it can also perform the fetch phase of the second instruction.

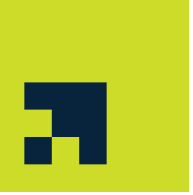


a. No pipelining



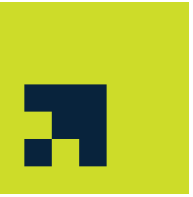
b. Pipelining

Parallel Processing



- Multiple control units, multiple arithmetic logic units and multiple memory units
- Multiple processors/cores work concurrently
- Flynn's Taxonomy:
 - SISD: Single Instruction, Single Data
 - SIMD: Single Instruction, Multiple Data
 - MISD: Multiple Instructions, Single Data
 - MIMD: Multiple Instructions, Multiple data
- Enables high-performance computing and multitasking

Multi-Core CPUs



- Integrate multiple processing cores on one chip (“CPU die”)
 - Increase concurrency and throughput
 - Increase efficiency by fetching data once
 - Increase power-efficiency (turning cores off, clocking them down)
- Architecture
 - Cores mostly isolated from one another
 - CPU cores may be identical, but also may be vastly different
 - Different ISAs for the cores, requires programs to run on the core it was created for
 - Busses and caches may be reused

Task 1



- Describe the instruction cycle with a labeled diagram

Task 2



- Compare the roles of the ALU, Control Unit, and Registers

Task 3



- Explain the differences between RISC and CISC architectures

Task 4



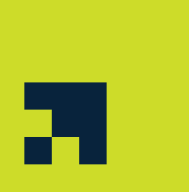
- What is the difference between the various types of RAM?

Task 5

- How does a CPU work?

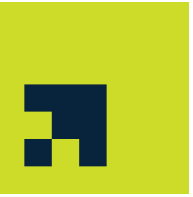


Task 6



- Explain how a CPU adds two values, one from an I/O device, one from the RAM, and stores the added value into a memory location on the SSD!

Task 7



- List common I/O devices and their interfacing mechanisms

Task 8



- What is pipelining and how does it improve CPU efficiency?

Task 9



- Discuss benefits and challenges of multicore processors